Vehicle Rental Management System

Object-Oriented Programming - Short Report

Student: Aws Hatem Thafer
Student ID: 12220122

1. Introduction and Program Structure

This project is a Java console program for a vehicle rental company. It manages cars, motorcycles, trucks, customers, rentals, returns, and maintenance. The user chooses an option from a menu that repeats until Exit is selected. The project uses ArrayList collections, so the data remains available only while the program is running.

I divided the program into classes with different responsibilities. Vehicle stores common vehicle information, and Car, Motorcycle, and Truck add their own details. Customer stores customer information. Rental connects a customer to a vehicle and records the cost and status. RentalSystem manages the collections and operations. RentalApp reads input and displays the menu, while Main only starts the application.

2. Encapsulation and Constructors

The fields are private to prevent direct access from other classes. Getters provide access to information, while update methods validate changes. For example, setDailyPrice() rejects a non-positive or non-finite price. Customer has methods to update the phone number and email. Vehicle controls availability through rent() and returnVehicle().

Parameterized constructors initialize objects with their required information. The keyword this refers to the current object. Subclasses use super(...) to call the Vehicle constructor. Customer also has two constructors: one accepts an email and the other starts with an empty email that can be updated later. The shorter constructor calls the longer one using this(...).

3. Abstraction: Abstract Class and Interface

Vehicle is abstract because the program should create a specific type of vehicle, rather than a general Vehicle object. It contains shared fields and implemented methods, such as calculateBaseCost(). It also declares calculateRentalCost() and displayInfo() as abstract methods, so each subclass must provide its own implementation.

Maintainable is an interface that declares performMaintenance() and needsMaintenance(). All three vehicle subclasses implement it. In this project, the abstract class shares state and common code, while the interface defines the maintenance operations that a class must provide. A Java class can extend one class and implement multiple interfaces. An interface does not hold per-object fields like the maintenance flag stored in Vehicle.

Polymorphism, Overriding and Overloading

4. Polymorphism Through the Vehicle Class

RentalSystem uses ArrayList<Vehicle> to store different vehicle types in one collection. A Vehicle reference can point to a Car, Motorcycle, or Truck object. When displayInfo() is called through that reference, Java runs the method belonging to the actual object. This is runtime polymorphism.

Rental calculates the cost by calling vehicle.calculateRentalCost(days, extraService). The same call works for all three vehicle types. Car adds $15 per day when insurance is selected. Motorcycle adds $5 per day when a helmet is selected. Truck adds one $50 service fee per rental. The program does not use a vehicle-type switch to calculate these costs.

The instanceof checks in RentalApp choose which optional-service question to ask. The checks in Rental update the motorcycle helmet state. These checks do not replace the overridden cost-calculation methods.

5. Interface-Based Polymorphism

The system also works with vehicles through Maintainable references. For example, performVehicleMaintenance() finds a Vehicle, casts it to Maintainable, and calls performMaintenance(). Java then selects the implementation provided by Car, Motorcycle, or Truck. The same interface reference can therefore be used with different vehicle types.

Each subclass reports its own maintenance checks. A car checks oil, brakes, and tires. A motorcycle checks the chain, brakes, and tires. A truck checks brakes, suspension, and the load platform. These are simulated checks shown as console messages. The shared completeMaintenance() method prevents maintenance during an active rental and clears the maintenance flag when the operation is allowed.

6. Method Overriding

Overriding means providing a new implementation of an inherited method with the same signature. Car, Motorcycle, and Truck override calculateRentalCost() and displayInfo() from Vehicle. They also implement the methods declared by Maintainable. The @Override annotation helps the compiler check that the methods match the inherited declarations.

7. Method and Constructor Overloading

Overloading means using the same name with different parameter lists. RentalSystem has displayVehicles() and displayVehicles(boolean availableOnly). The first displays all vehicles. The second can display only vehicles that are available and do not need maintenance. The no-argument version calls the boolean version with false to reuse the same code.

The same idea is used in displayRentals() and displayRentals(boolean activeOnly). Customer demonstrates constructor overloading with and without an email parameter. Overload selection happens at compile time using the method name and argument types, while overridden method selection happens at runtime using the actual object. These examples show both compile-time and runtime polymorphism, including runtime dispatch through an interface.

Associations, Validation and Testing

8. Associations, Collections, Static and Final

Rental stores references to one Customer and one Vehicle. A customer can have several rentals, and a vehicle can appear in several past rentals, but it can have only one active rental at a time. RentalSystem stores the main objects in three ArrayList collections and searches these collections by ID.

The static nextRentalId field is shared by all Rental objects and generates IDs during the current run. The fee constants use static final because their values are shared and should not change. For example, SERVICE_FEE is fixed at $50. Each rental saves its total cost when created, so changing a vehicle price later does not change an earlier transaction.

9. Validation and Maintenance Rules

The program rejects duplicate vehicle and customer IDs, unknown customers or vehicles, non-positive rental days, and attempts to rent an unavailable vehicle. Completing a rental makes the vehicle available again, and completing it twice is rejected. RentalApp catches invalid input and operation exceptions, displays a message, and returns to the menu.

Availability and maintenance are separate flags. A vehicle may be idle but still require maintenance. Such a vehicle cannot be rented. Maintenance is also blocked while the vehicle is rented. Revenue is calculated from completed rentals only; active rentals do not contribute to the total.

10. Manual Test Results

Test performed	Observed result
Car: $60/day, 4 days, with insurance	$300.00; rental active
Motorcycle: $30/day, 2 days, with helmet	$70.00; rental active
Truck: $100/day, 3 days	$350.00; rental active
Return car, then return the same rental again	First return accepted; second rejected
Return car and truck; motorcycle still active	Total revenue: $650.00
Rent a vehicle flagged for maintenance	Rejected
Perform maintenance on rented motorcycle	Rejected
Duplicate vehicle ID; unknown customer or vehicle	Rejected
Zero or negative rental days	Rejected

11. Conclusion

The project helped me practice organizing a program into related classes. Inheritance reused common vehicle code, while abstract methods and the maintenance interface allowed different behavior through shared references. The manual tests checked rental costs, returns, revenue, and important rejection rules. The program is limited to a console interface and does not save data after closing, as file persistence is not required.